

121. Best Time to Buy and Sell Stock Easy

Acceptance: 56.2% | [LeetCode](#)

You are given an array `prices` where `prices[i]` is the price of a given stock on the `i`th day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return `0`.

Example 1:

```
**Input:** prices = [7,1,5,3,6,4]
**Output:** 5
**Explanation:** Buy on day 2 (price = 1) and sell on
```

day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

```
**Input:** prices = [7,6,4,3,1]
**Output:** 0
**Explanation:** In this case, no transactions are done and the max profit = 0.
```

Constraints:

- `1 <= prices.length <= 105`
- `0 <= prices[i] <= 104`

Claude Opus Solution

Best Time to Buy and Sell Stock

1. Intuition

The core constraint is: **you must buy before you sell**. This means for any selling day, you want to have bought at the lowest price seen *on or before* that day.

Key Insight: As we scan through prices left to right, we only need to track the **minimum price seen so far**. For each price, the best possible profit if we sell *today* is `current_price - min_price_so_far`. We just keep the maximum of all these daily-best profits.

This transforms a seemingly $O(n^2)$ "check all buy-sell pairs" problem into a single $O(n)$ pass by realizing we never need to look back—we just carry the minimum forward.

2. Approach (Pseudocode)

1. Initialize `min_price = infinity` (lowest buy price seen so far)
2. Initialize `max_profit = 0` (best profit found; 0 if no profit possible)
3. For each price in prices:
 - a. Update `min_price = min(min_price, price)`
→ ensures we always "bought" at the cheapest valid day
 - b. Compute profit if we sold today = `price - min_price`
 - c. Update `max_profit = max(max_profit, profit)`
4. Return `max_profit`

3. Python Solution

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        # min_price tracks the lowest price seen so far (best day to have bought).
        # Start at infinity so the first price always becomes the initial minimum.
        min_price = float("inf")

        # max_profit holds the best profit found.
        # Stays 0 if prices only decline (no profitable transaction).
        max_profit = 0

        # Single left-to-right pass. By scanning forward, any price we consider
        # selling at is guaranteed to come AFTER the min_price we bought at.
        for price in prices:
            # Step 1: Update the cheapest buying price seen so far.
            # If today is cheaper, it becomes our candidate buy day.
            min_price = min(min_price, price)
```

```

# Step 2: Compute the profit if we sold at today's price,
# having bought at the cheapest earlier (or current) day.
# Then keep the maximum profit observed.
max_profit = max(max_profit, price - min_price)

return max_profit

```

Note on `min_price`: When `price == min_price` (e.g., a new minimum on the same iteration), `price - min_price = 0`, which correctly contributes no profit. This elegantly handles the "buy and sell same day" edge naturally yielding 0.

4. Complexity Analysis

Metric	Complexity	Explanation
Time	$O(n)$	Single pass through the array; each element processed once with $O(1)$ work.
Space	$O(1)$	Only two scalar variables (<code>min_price</code> , <code>max_profit</code>) regardless of input size.

5. Alternative Approaches

A) Two-Pointer (Sliding Window) — `solution1.py`

- `left` = buy pointer, `right` = sell pointer.
- If `prices[left] < prices[right]`, record the gap and advance `right`.
- Else, the current price is a new minimum, so jump `left` to `right`.
- Same $O(n)$ / $O(1)$ complexity, but slightly more verbose and harder to reason about edge cases.

B) Brute Force (for understanding only)

```

# Check every (buy, sell) pair where buy < sell
max_profit = 0
for i in range(len(prices)):
    for j in range(i + 1, len(prices)):
        max_profit = max(max_profit, prices[j] - prices[i])
return max_profit

```

- **Time: $O(n^2)$, Space: $O(1)$** — too slow for $n = 10^5$ (TLE). Useful only to grasp the problem.

C) Dynamic Programming View

Define `dp[i]` = max profit ending at day `i`. The recurrence is essentially: - `min_price = min(min_price, prices[i])` - `dp[i] = prices[i] - min_price`

This is exactly the optimal solution above, just framed as DP—confirming the greedy "track minimum" strategy is provably optimal.

🔗 Pattern Takeaway

This is the classic "**running minimum/maximum**" pattern. Whenever a problem asks for the best pair `(i, j)` with `i < j` under some difference/comparison, consider whether tracking a single running extreme as you scan eliminates the need for nested loops.

Codex Solution

1. Intuition

We need to buy **before** we sell, and we want the maximum difference:

```
sell_price - buy_price
```

For each day, if we pretend to sell on that day, the best possible buy day must be the **lowest price seen before or on that day**.

So while scanning from left to right, we keep track of:

1. `min_price` — the cheapest price seen so far
2. `max_profit` — the best profit found so far

At each price:

- If this price is lower than `min_price`, update `min_price`
- Otherwise, calculate the profit if we sell today
- Update `max_profit` if this profit is better

This is a classic **one-pass running minimum** pattern.

2. Approach

Pseudocode

```
function maxProfit(prices):
    min_price = infinity
    max_profit = 0

    for price in prices:
        if price < min_price:
            min_price = price

        current_profit = price - min_price

        if current_profit > max_profit:
            max_profit = current_profit

    return max_profit
```

3. Python Solution

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        # min_price stores the lowest stock price seen so far.
        # We initialize it to infinity so that the first price will replace it.
        min_price = float("inf")

        # max_profit stores the best profit we can achieve.
        # If no profitable transaction exists, it remains 0.
        max_profit = 0

        # Iterate through each day's stock price from left to right.
        for price in prices:
            # Update the minimum buying price seen so far.
            # This ensures we always buy before or on the current day.
            min_price = min(min_price, price)

            # Calculate the profit if we sell at today's price.
            current_profit = price - min_price

            # Update the maximum profit if selling today is better.
            max_profit = max(max_profit, current_profit)

        # Return the best profit found.
        return max_profit
```

4. Complexity Analysis

Time Complexity: $O(n)$

We iterate through the `prices` array exactly once, where `n` is the number of days.

Space Complexity: $O(1)$

We only use two extra variables:

```
min_price
max_profit
```

So the space usage is constant.

5. Alternative Approaches

Alternative 1: Two Pointers

We can use two pointers:

- `left` = buy day
- `right` = sell day

If `prices[right] > prices[left]`, calculate profit.

If `prices[right] <= prices[left]`, move `left` to `right`, because we found a better buying price.

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        left = 0    # Buy day
        right = 1   # Sell day

        max_profit = 0

        while right < len(prices):
            # If selling price is higher than buying price,
            # calculate the profit.
            if prices[right] > prices[left]:
                profit = prices[right] - prices[left]
                max_profit = max(max_profit, profit)
            else:
                # Found a cheaper buying price.
                left = right

            # Move sell day forward.
            right += 1

        return max_profit
```

Time Complexity: $O(n)$

Space Complexity: $O(1)$

Alternative 2: Brute Force

Try every possible pair of buy and sell days.

```
for buy in range(n):
    for sell in range(buy + 1, n):
        profit = prices[sell] - prices[buy]
```

This works logically, but it is too slow.

Time Complexity: $O(n^2)$

Space Complexity: $O(1)$

Since `prices.length` can be up to 10^5 , an $O(n^2)$ solution would likely time out.

Key Pattern

This problem is an example of:

```
Track the best previous value while scanning left to right.
```

Here, the "best previous value" is the minimum price seen so far. Then for each current price, we compute the best profit if we sell today.